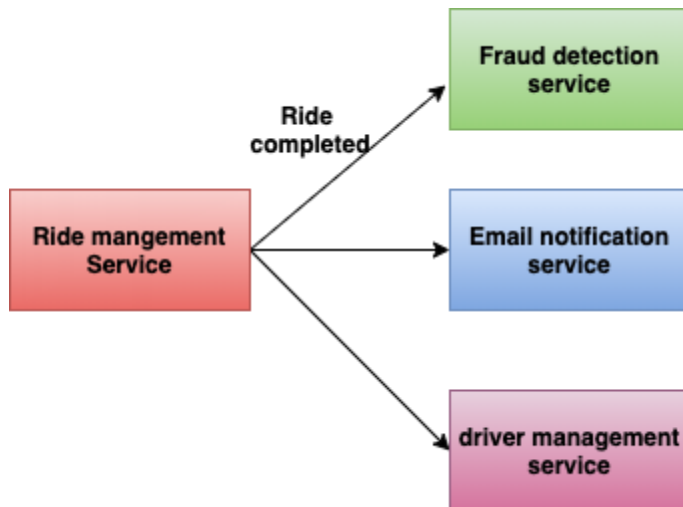


Apache Kafka

Large companies contain hundreds of microservices, all needing data to operate. When we talk about technologies that they use, we come across Apache Kafka, which is used to handle trillions of data per day. In this blog, let's understand what Kafka is and why it is so popular.

Let's take an example of microservices involved in ride application. When a ride is completed, this information is needed by multiple microservices.

- Ride management service manages ride information.
- Email notification service sends a ride receipt in the email.
- Driver management service manages driver information. It updates the status of the driver so that he can be allocated for the next ride.
- Fraud detection service detects any fraudulent activity that had happened in the ride.



Ride management microservices

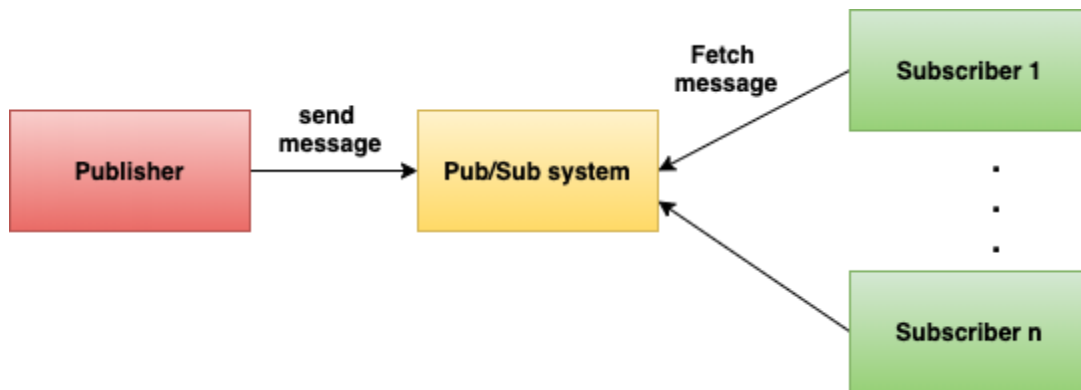
In this case, we have many services that need ride-completion data to operate on. If synchronous calls are made to all services then we will end up making this request slower. And also it will introduce coupling between services.

We can make calls asynchronous by using a background job processor like Sidekiq or Go-craft. In this case, we need to add jobs in ride-management-service. Whenever a new service wants ride-completion data, we need to do code changes in ride-management-service to add new jobs. This won't solve the above problem of coupling between services.

The next approach is to use some queue like RabbitMQ. In this case, only one consumer can consume the data from the queue. In the above scenario, we have many consumers who need ride-completion data. So simple queue won't solve our problem.

How nice would it be if we have a publisher-subscriber system here?

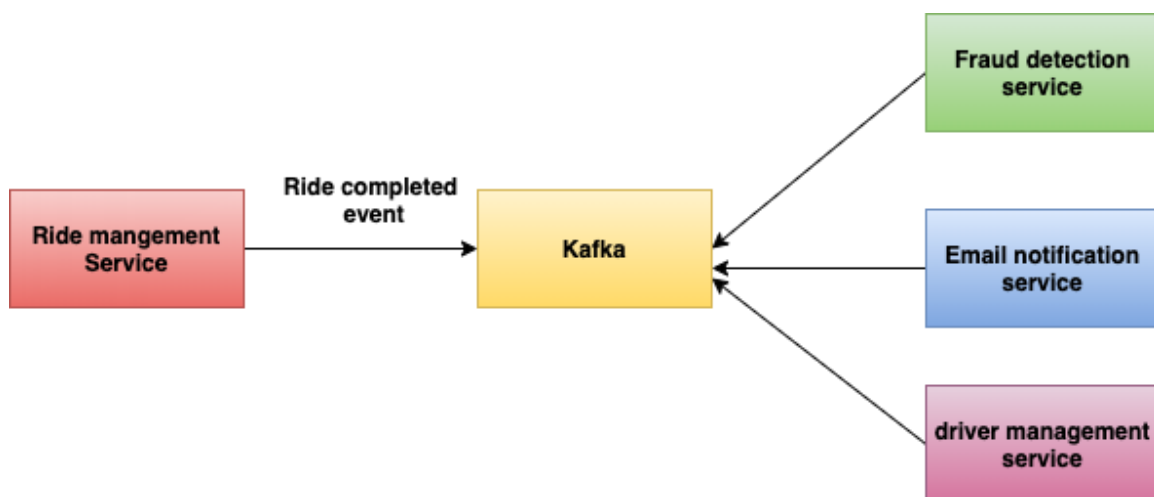
Press enter or click to view image in full size



Pub/Sub system

Whenever an event happens, the publisher publishes messages to the messaging system. Any microservice that needs this information can subscribe to it. This type of architecture is called event-driven architecture. It provides multiple benefits:

- Loosely coupled microservices
- Asynchronous communication
- Easily scalable since services are decoupled.



Ride management microservices with Kafka

Apache Kafka is one such publisher-subscriber messaging system. We can have multiple consumers who can read the same events. The data sent to Kafka is persisted to some duration defined by the retention policy.

In Kafka, publishers are called producers. And subscribers are called consumers. In the above example, when a ride gets completed, producers can publish this event to Kafka, and all other services which require this event can subscribe to it. But where and how does Kafka store these events?

1. Introduction to Apache Kafka

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications. It allows systems to publish, store, and process streams of records in a fault-tolerant and scalable way. Kafka is widely used in microservices architecture, log aggregation, messaging systems, and event-driven systems.

In simple terms, Kafka acts as a central backbone where producers send events/messages and consumers read and process them.

Example Use Case:

- Order Service publishes order events
- Payment Service consumes order events
- Notification Service also consumes the same events

2. Kafka Core Components

Kafka Cluster

A Kafka cluster is a group of multiple Kafka brokers working together to provide scalability, high availability, and fault tolerance.

Broker

The place where topics are kept and maintained are called [Brokers/ Kafka servers](#). Kafka broker receives messages from producers and stores them on disk. When a consumer wants to read from a topic then it must establish a connection with a broker. Upon the connection, consumers can read from the beginning of the topic or from the time connection is made. **Topic**

In Kafka, the specific location where messages are stored is called [Topic](#). If we have to compare it with the SQL database then the topic is like a table and the message is like a row of data. But in Kafka, Message is an immutable record which means that once they are received into a topic they can not be changed. Each message that is sent is appended to a time-ordered sequence. This style of maintaining data as events is an architectural style known as [event sourcing](#)

Partition

Partitions divide a topic into smaller units to allow parallel processing and scalability. Each partition maintains message order.

Producer

Producers are applications that publish messages to Kafka topics.

Consumer

Consumers read messages from Kafka topics and process them.

Consumer Group

A consumer group is a set of consumers working together to consume messages from a topic in parallel.

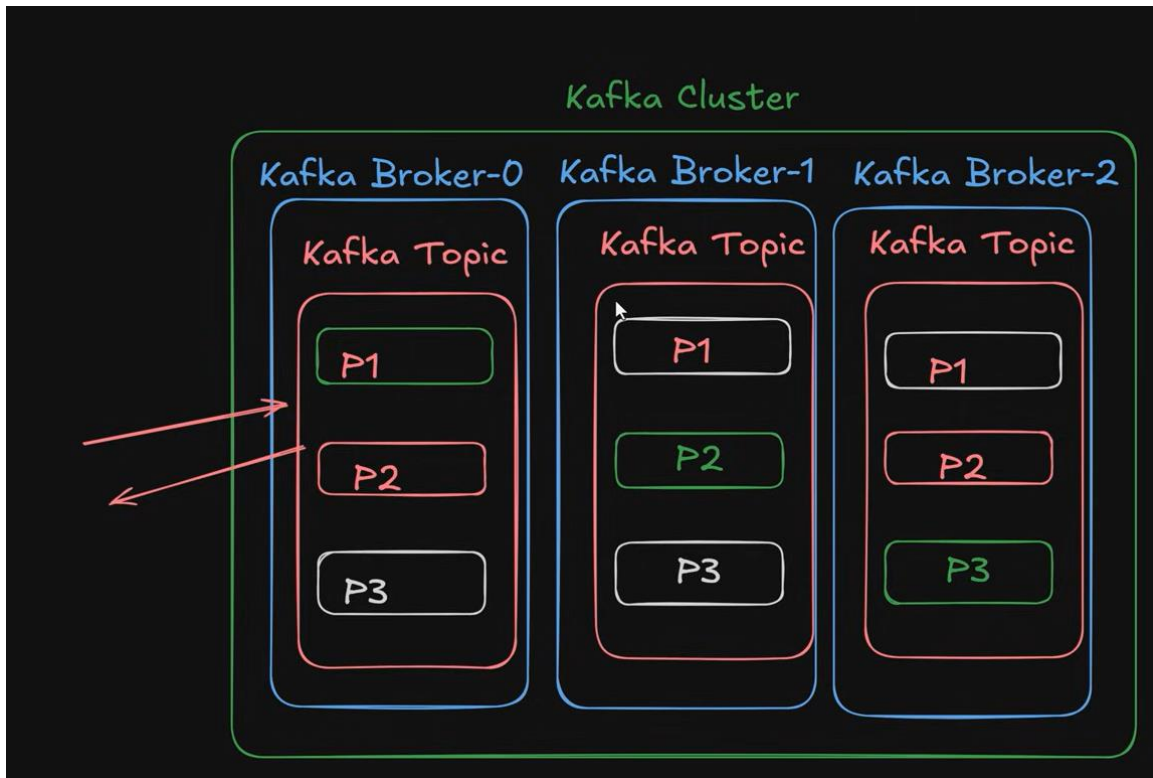
ZooKeeper / KRaft

ZooKeeper was traditionally used for broker coordination, metadata management, and leader election. Modern Kafka uses KRaft mode which removes ZooKeeper and manages metadata internally.

3. Kafka Working Mechanism

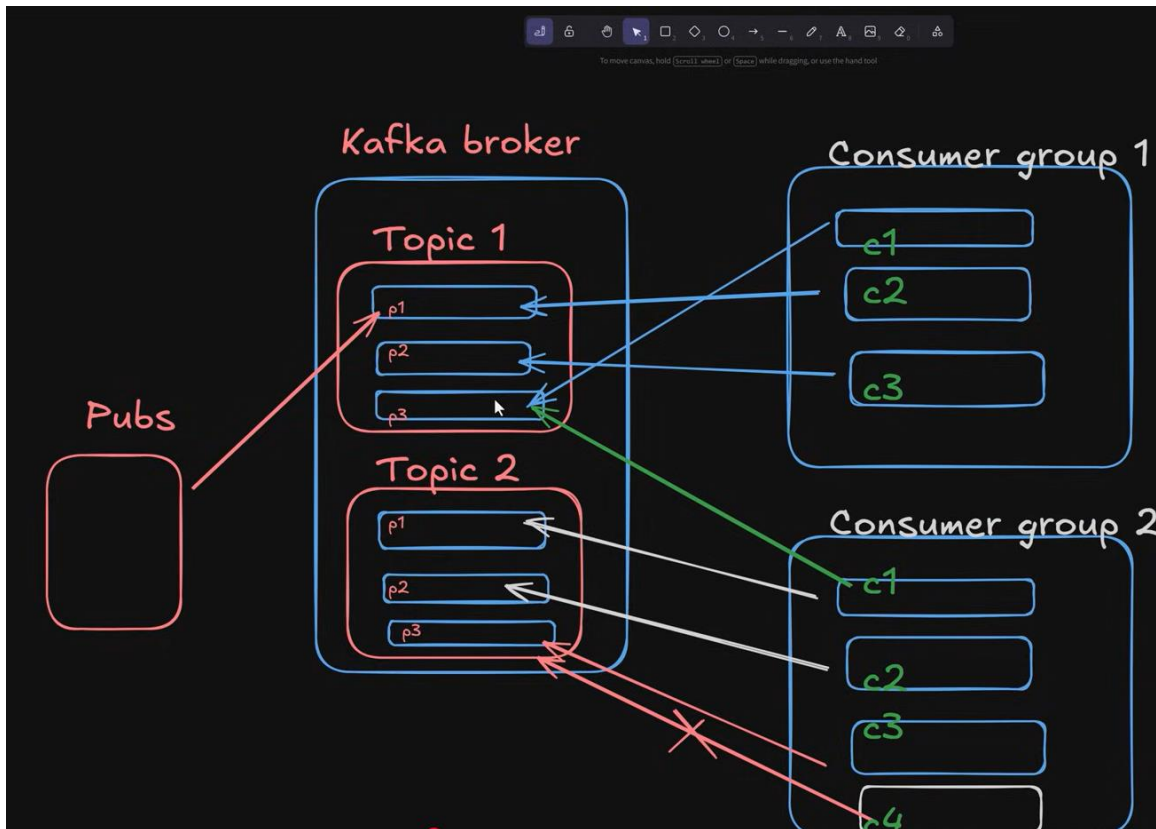
1. Producers send messages to topics.
2. Topics are divided into partitions stored across brokers.
3. Brokers manage storage and replication.
4. Consumers subscribe to topics and read messages.
5. Consumer groups allow parallel consumption.
6. Offsets track message consumption progress.

4. Visual Explanation Based on Provided Diagrams



Kafka Cluster and Broker with Partitions

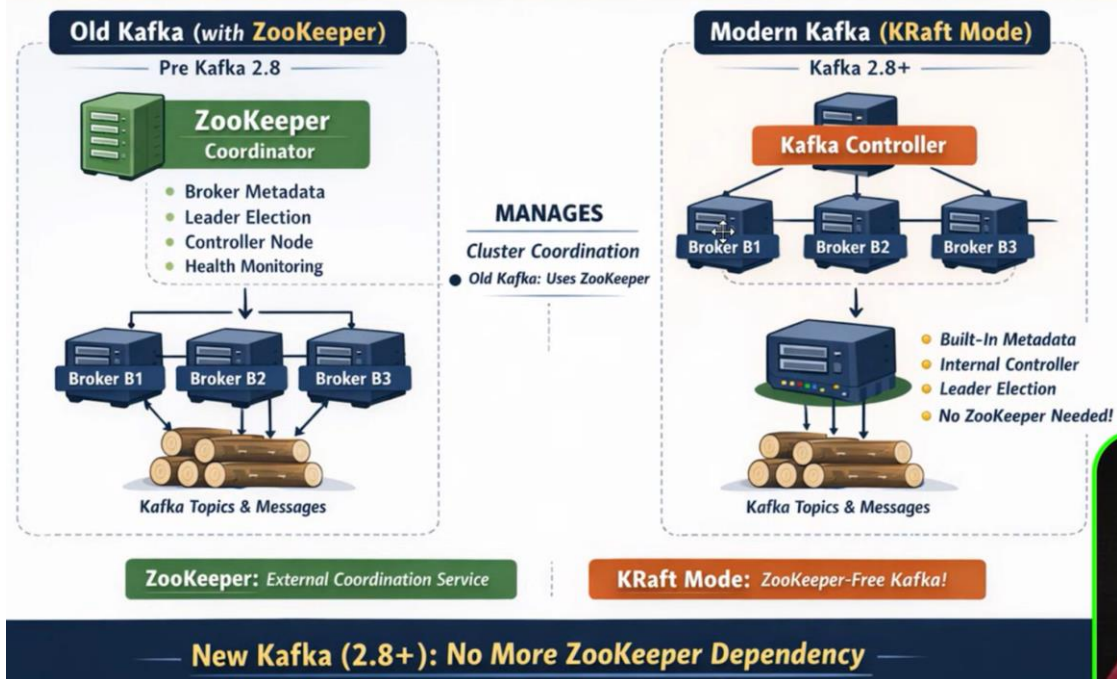
A Kafka cluster contains multiple brokers. Each broker stores topic partitions. Partitions are distributed across brokers for scalability and fault tolerance.



Producers, Topics, and Consumer Groups

Producers publish messages to topics which are divided into partitions. Each partition in a consumer group is consumed by only one consumer within the same group, but multiple consumer groups can consume the same partition independently.

What is ZooKeeper in Kafka?



ZooKeeper vs KRaft Mode

Older Kafka versions used ZooKeeper for coordination, leader election, and cluster metadata management. Modern Kafka uses KRaft mode which provides internal metadata management and removes the need for ZooKeeper.

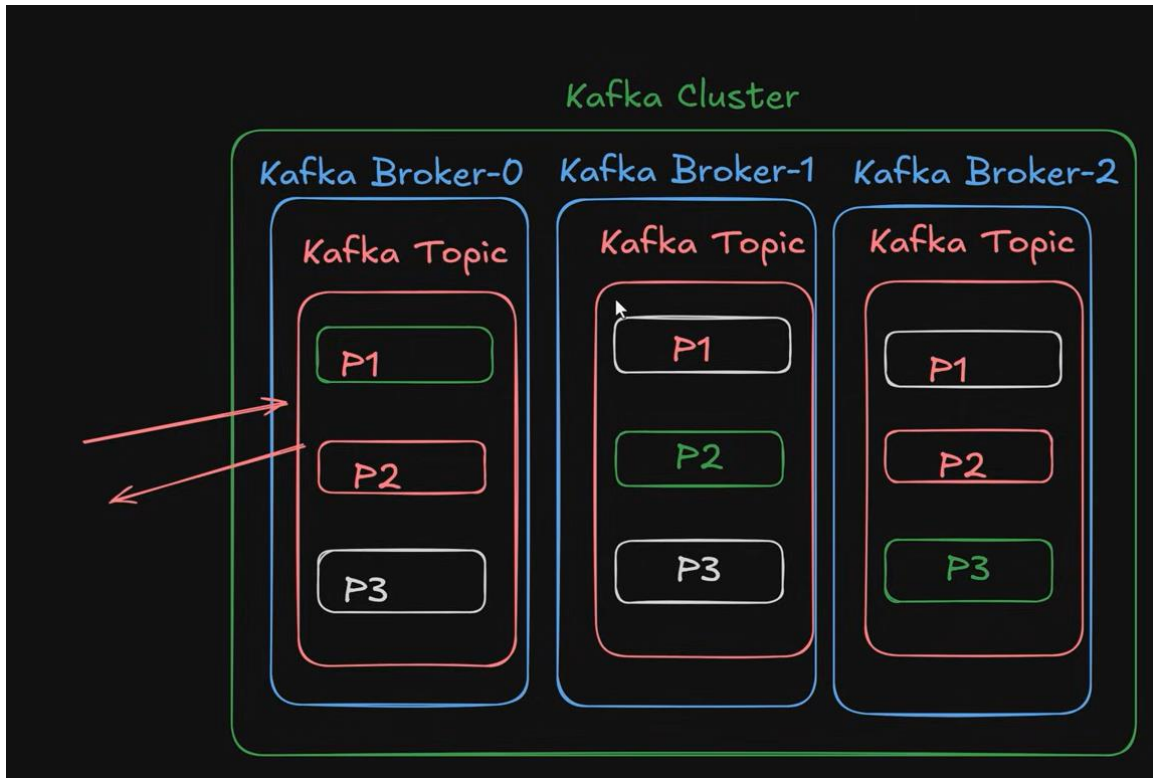
Fault tolerance in Apache Kafka

What happens when the broker/server goes down? Is all the data lost?

Partition Replication

Fault tolerance in Kafka is done by copying the partition data to other brokers which are known as **replicas**. There is a configuration that specifies how many copies of the partition you need. Its called a **replication factor**.

Each broker will hold one or more partitions. And each of these partitions can either be a replica or leader for the topic. All the writes and reads to a topic go through the leader and the leader coordinates to update replicas with new data.



In this image **green** colour is **Leader** and **white** colour is **followers**

In the above example, there are three brokers. For **partition-1**, **Broker-1** is a leader. **Broker-2** and **Broker-3** are the replica brokers.

So what ever Leader is doing same will be reflected in the followers the same things will be reflected in the followers

Consumers will always deal with the leaders not with the brokers
think like because of some reasons the Broker-1 stops working so in that case any one of the followers act as leader and prevents the data loss and flow

Who decides the leaders i.e Zookeeper means Zookeeper is the person who will decides the leaders nothing but leader election

So in real time applications it will we multiple brokers/servers are maintained not one broker

Step-by-Step Spring Boot Kafka Setup

✓ STEP 1 — Add Dependency

Maven

```
<dependency>

  <groupId>org.springframework.kafka</groupId>

  <artifactId>spring-kafka</artifactId>

</dependency>
```

That's it 👍

✓ STEP 2 — application.yml Config

Same like RabbitMQ host/port — but Kafka uses **bootstrap-servers**.

spring:

kafka:

bootstrap-servers: localhost:9092

consumer:

group-id: my-group

auto-offset-reset: earliest

key-deserializer: org.apache.kafka.common.serialization.StringDeserializer

value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

producer:

key-serializer: org.apache.kafka.common.serialization.StringSerializer

value-serializer: org.apache.kafka.common.serialization.StringSerializer

No username/password normally (unless secured cluster).

✓ STEP 3 — Topic Configuration (Optional but Recommended)

Equivalent to:

👉 Queue + Exchange creation in RabbitMQ.

@Configuration

```
public class KafkaTopicConfig {  
  
    @Bean  
    public NewTopic orderTopic() {  
        return TopicBuilder.name("order-topic")  
            .partitions(3)  
            .replicas(1)  
            .build();  
    }  
}
```

Spring will auto-create topic if broker allows.

✓ STEP 4 — Producer Service (RabbitTemplate Equivalent)

RabbitMQ:

RabbitTemplate.convertAndSend()

Kafka:

KafkaTemplate.send()

@Service

```
public class KafkaProducerService {  
  
    @Autowired
```

```
private KafkaTemplate<String, String> kafkaTemplate;

public void sendMessage(String message) {
    kafkaTemplate.send("order-topic", message);
}
}
```

✓ STEP 5 — Consumer Listener (@RabbitListener Equivalent)

RabbitMQ:

@RabbitListener

Kafka:

@KafkaListener

@Service

```
public class KafkaConsumerService {
```

```
    @KafkaListener(topics = "order-topic", groupId = "my-group")
    public void listen(String message) {
        System.out.println("Received message: " + message);
    }
}
```

Done 📄 that's your consumer.

✓ STEP 6 — Controller (Test API)

@RestController

@RequestMapping("/kafka")

```
public class KafkaController {
```

@Autowired

private KafkaProducerService producerService;

@GetMapping("/send")

public String send(@RequestParam String msg) {

producerService.sendMessage(msg);

return "Message sent";

}

}

Test:

<http://localhost:8080/kafka/send?msg=hello>

□ Full Flow (Exactly Like RabbitMQ Thinking)

Controller

↓

KafkaTemplate.send()

↓

Kafka Broker (Topic + Partition)

↓

@KafkaListener Consumer

□ First — Big Concept Difference

RabbitMQ Client Config = Connection Settings

You mainly configure:

- where broker is
- how to connect
- queue binding

Message consumption behaviour mostly server-driven.

Kafka Client Config = Connection + Consumption Behaviour

Kafka needs:

- broker location
- how to read messages
- how to serialize data
- where to start reading (offset logic)
- consumer group behaviour

Because Kafka consumers control **how they read data**.

Now Your Config Explained

bootstrap-servers

bootstrap-servers: localhost:9092

 Equivalent to RabbitMQ:

host + port

But Kafka has **many brokers**.

Example:

bootstrap-servers: localhost:9092,localhost:9093

Spring connects to one → fetches full cluster metadata.

So bootstrap server = **entry point to cluster**.

✓ **2** `consumer.group-id`

group-id: my-group

🔗 RabbitMQ equivalent = multiple consumers on same queue.

But Kafka uses **consumer groups** for:

- load balancing
- parallel consumption
- fault tolerance

Example:

Topic has 3 partitions

Group has 3 consumers

→ each gets 1 partition

Without group-id → Kafka treats consumer as independent.

✓ **3** `auto-offset-reset`

auto-offset-reset: earliest

VERY IMPORTANT concept — RabbitMQ does NOT have this.

Kafka stores:

offset = last consumed position

When consumer starts FIRST TIME:

Value Meaning

earliest read from beginning

latest read only new messages

Example:

Topic has 100 old messages

Consumer starts

earliest → reads all 100

latest → reads only future ones

✓ 4 key-deserializer

key-deserializer: StringDeserializer

Kafka stores data as **bytes**.

When consuming:

byte[] → String

byte[] → JSON → Object

Deserializer converts Kafka bytes → Java object.

RabbitMQ usually handles conversion internally or via converter.

✓ 5 value-deserializer

Same concept but for message body.

✓ 6 producer.key-serializer

StringSerializer

Producer sends:

Java object → bytes

Serializer converts:

String → byte[]

JSON → byte[]

DTO → byte[]

RabbitMQ hides this mostly.

Kafka makes it explicit.

✓ `producer.value.serializer`

Same but for message content.

📊 Why Kafka Needs More Config Than RabbitMQ

Concept	RabbitMQ	Kafka
Offset tracking	✗ No	✓ Yes
Replay messages	✗ No	✓ Yes
Consumer positioning	✗ No	✓ Yes
Partition routing	✗ No	✓ Yes
Serialization control	Mostly hidden	Explicit
Consumer groups	Basic	Core architecture

🔗 Real Production Kafka Config Adds Even More

You'll later see:

`enable-auto-commit`

`max-poll-records`

`session-timeout-ms`

`fetch-min-bytes`

`acks`

`retries`

`batch-size`

`linger-ms`

These control:

- throughput
- reliability
- batching
- retry logic

☑️ How to give multiple group IDs in Kafka ?

👉 first understand one important thing:

👉 **group-id is NOT global for the application**

👉 it is **per consumer (listener)**

means:

- one listener → one group
- another listener → another group

you don't define multiple groups in single yml property normally.

💎 Method 1 — override groupId in @KafkaListener (MOST COMMON)

yml

spring:

kafka:

bootstrap-servers: localhost:9092

consumer:

group-id: default-group

listener 1

```
@KafkaListener(topics = "order-topic", groupId = "group-A")
```

```
public void listenA(String msg){
```

```
System.out.println("Group A -> " + msg);  
}
```

listener 2

```
@KafkaListener(topics = "order-topic", groupId = "group-B")  
  
public void listenB(String msg){  
  
    System.out.println("Group B -> " + msg);  
}
```

👉 result:

- both groups receive message
 - each group has its own offset
 - independent consumption
-

💎 **Method 2 — separate consumer factories (advanced)**

used when:

- different deserializers
- different configs
- different security

but 90% projects use `@KafkaListener(groupId="")`

✅ **2 Partitions & Replicas — VERY IMPORTANT CONCEPT**

💎 **Partitions**

topic → divided into partitions

example:

topic = orders

partitions = 3

orders-0

orders-1

orders-2

why ?

👉 parallel processing

👉 scaling

👉 load distribution

◆ Replicas — YES 👉 followers

you are correct.

replication = copies of partition

example:

partition 0

leader -> broker1

follower -> broker2

follower -> broker3

◆ Roles

□ Leader

- handles read
- handles write
- producer sends here
- consumer reads here

● Followers

- copy data from leader

- passive
 - for fault tolerance
-

◆ What happens if leader dies ?

Kafka automatically:

👉 promotes one follower → new leader

this is why Kafka is fault tolerant.

◆ Example while creating topic

```
kafka-topics.sh \  
--create \  
--topic orders \  
--partitions 3 \  
--replication-factor 2 \  
--bootstrap-server localhost:9092
```

means:

- 3 partitions
 - each partition has 2 copies
 - 1 leader
 - 1 follower
-

✓ 1 Visual Understanding

partition + replica structure

Broker1:

orders-0 (leader)

orders-1 (follower)

Broker2:

orders-0 (follower)

orders-1 (leader)

load balancing + failover 👍

✓ 4 Consumer Groups vs Partitions (VERY IMPORTANT INTERVIEW)

if:

partitions = 3

group consumers = 3

each consumer gets one partition

C1 -> P0

C2 -> P1

C3 -> P2

if:

consumers > partitions

some consumers idle

if:

consumers < partitions

consumer handles multiple partitions

✓ 5 Big Difference From RabbitMQ

Feature Kafka

RabbitMQ

Feature	Kafka	RabbitMQ
group id	consumer scaling	no concept
partition	parallel processing	queue based
replica	fault tolerance	mirrored queues
offset	consumer controlled	broker controlled

ahh good 👍 now you're going into **multi-broker / multi-cluster Kafka** — nice level up.

first small correction:

👉 **topics are NOT created “in a server” normally**

👉 topics belong to a **Kafka cluster**

👉 brokers inside the cluster store partitions

so there are **2 different cases** — you need to know which one you mean.

✅ □ CASE 1 — Same Kafka Cluster (multiple brokers)

example:

Broker1 → 9092

Broker2 → 9093

Broker3 → 9094

when you create:

@Bean

```
public NewTopic orderTopic() {
    return TopicBuilder.name("order-topic")
        .partitions(3)
        .replicas(2)
```

```
        .build();  
    }
```

☞ Kafka automatically distributes partitions across brokers.

you DO NOT choose server manually.

example distribution internally:

order-topic-0 -> broker1 (leader)

order-topic-1 -> broker2 (leader)

order-topic-2 -> broker3 (leader)

so answer:

☞ you cannot say "this topic only on broker1"

☞ Kafka manages distribution automatically.

☑ □ CASE 2 — Different Kafka Clusters (Different Servers)

THIS is probably what you mean.

example:

Cluster A → localhost:9092

Cluster B → localhost:9095

☞ now you need **multiple Kafka configurations** in Spring Boot.

◆ Step 1 — application.yml

kafka:

cluster1:

bootstrap-servers: localhost:9092

cluster2:

bootstrap-servers: localhost:9095

◆ Step 2 — Config for Cluster 1

@Bean

```
public KafkaAdmin kafkaAdmin1() {  
    Map<String, Object> configs = new HashMap<>();  
    configs.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");  
    return new KafkaAdmin(configs);  
}
```

@Bean

```
public NewTopic topicCluster1() {  
    return TopicBuilder.name("topic-cluster1")  
        .partitions(2)  
        .replicas(1)  
        .build();  
}
```

◆ Step 3 — Config for Cluster 2

@Bean

```
public KafkaAdmin kafkaAdmin2() {  
    Map<String, Object> configs = new HashMap<>();  
    configs.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9095");  
    return new KafkaAdmin(configs);  
}
```

👉 but important:

Spring Boot default topic creation works only with **one KafkaAdmin**.

for multiple clusters → you usually:

- create topics manually
 - OR use AdminClient manually
-

◆ Step 4 — Manual Topic Creation Using AdminClient

```
Properties props = new Properties();
```

```
props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9095");
```

```
try (AdminClient adminClient = AdminClient.create(props)) {
```

```
    NewTopic topic = new NewTopic("topic-cluster2", 3, (short)1);
```

```
    adminClient.createTopics(Collections.singleton(topic));
```

```
}
```

✓ □ REAL WORLD ARCHITECTURE

normally companies:

- 1 cluster per environment
 - dev cluster
 - staging cluster
 - prod cluster

NOT:

✗ topic per server

✗ manual broker mapping

🔗 VERY IMPORTANT THING YOU SHOULD KNOW

this is WRONG thinking in Kafka:

topic → server

correct thinking:

topic → cluster

cluster → multiple brokers

broker → partitions